

BookMyShow — CAP Theorem and Design Trade-Offs

1. CAP Theorem Application in BookMyShow

In a distributed ticket booking system like BookMyShow, the CAP theorem (Consistency, Availability, Partition Tolerance) cannot be applied uniformly across the entire platform. Network partitions in distributed environments are inevitable; therefore, the system is designed with deliberate trade-offs per subsystem:

- **Consistency over Availability (CP):** Critical for **Seat Booking** and **Payment** workflows. Allowing two users to concurrently reserve or purchase the same physical seat (double-booking) is a catastrophic failure. If a partition occurs, the booking service must reject or delay conflicting writes rather than accept conflicting data.
- **Availability over Consistency (AP):** Preferred for **Search & Discovery**, **Movie Catalogs**, and **Notifications**. When users browse available films or showtimes, high availability and low latency (< 100 ms) are essential. A slight delay in seeing newly scheduled shows (eventual consistency) is acceptable and will not cause operational corruption.

2. Subsystem CAP Trade-Off Summary Table

Subsystem	Priority	Trade-off Decision	Notes
Seat Booking	Consistency > Availability	Lock-based reservation	Prevent double booking
Search	Availability > Consistency	Eventual consistency via cache refresh	Faster responses
Payment	Consistency > Availability	Idempotent operations	No double charge
Notifications	Availability > Consistency	Retry on failure	Best-effort delivery

3. Failure Handling & Trade-Off Approaches

1. Seat Reservation Conflicts

- **Approach:** Distributed locking combined with conditional database updates.
- **Mechanism:** When multiple users simultaneously select the same seat, Redis locks with short TTLs and atomic conditional queries (e.g., `WHERE status = 'AVAILABLE'`) ensure only the first request succeeds.
- **Trade-off:** Conflicting requests fail fast with `409 Conflict` , sacrificing availability for those specific requests to maintain strict consistency.

2. Payment System Failure

- **Approach:** Automated lock expiration with timeout-driven release.
- **Mechanism:** Seats are held in a `HELD` state for a fixed window (e.g., 8 minutes). If the external payment provider times out or crashes during checkout, a background worker or TTL resets the seat status to `AVAILABLE`.
- **Trade-off:** The user's checkout session is terminated safely without holding inventory indefinitely, ensuring other users can book the seats.

3. Cache Invalidation Delay

- **Approach:** Event-driven cache eviction with authoritative database verification.
- **Mechanism:** Search endpoints read movie listings and show schedules directly from Redis caches. When updates occur, invalidation events purge stale cache keys.
- **Trade-off:** Search queries may temporarily display show timings that lag by a few seconds. However, real-time seat availability is verified against the primary database when entering checkout.

4. Message Queue Retry and Duplication

- **Approach:** At-least-once delivery coupled with idempotent consumers.
- **Mechanism:** Kafka ensures messages are not lost across partition failures by retrying unacknowledged messages. Consumer services track processed event IDs in a deduplication cache.
- **Trade-off:** Potential message redelivery is accepted in exchange for guaranteed delivery, while consumer idempotency prevents duplicate notifications or processing errors.